A sync bug that erased the data it had just refused to read

99.3% of records were rejected on arrival. The app then acknowledged the delete, so the strap wiped its own buffer. Neither bug was interesting on its own. Stacked, they made the loss unrecoverable.

16 AUG 2026 · 9 MIN READ

A personal health app pulled heart-rate history off a wrist-worn strap over Bluetooth Low Energy. The backfill path rejected 99.3% of the records it had just asked for, and then told the strap those records were safe to delete. The strap complied. That combination turned a sync bug into permanent data loss, and it ran that way for months without a single error surfacing to the screen.

Nothing in this post is exotic. The two defects are the kind that pass code review. What makes it worth writing down is the interaction: an ingest bug and an acknowledgement bug that are each survivable alone, and destructive together.

How the transfer is supposed to work

The strap keeps a rolling on-device buffer, roughly 72 hours deep. When the phone connects, it asks for history. The strap replays from its oldest unacknowledged record forward. The app stores what it receives, then sends an acknowledgement so the strap can trim the buffer and reclaim space for new samples.

That handshake is the whole safety property. The trim is destructive and irreversible on the device side, so the acknowledgement is a promise: this data is durable somewhere else now, you may forget it.

Bug one: an accept window measured in minutes

The app validated every incoming record against a gap window before storing it. The intent was reasonable, which is why it survived: only accept records that fall inside the period the app believes it is missing.

The window was minutes wide. The strap serves up to 72 hours. So a record that was 69 hours old, exactly the kind of record backfill exists to recover, fell outside the window and was marked `out_of_range`. Not some records. Effectively all of them. The mechanism rejected historical data by design, and history was the only thing it was ever asked to fetch.

Measured across real sessions, the failure rate was 99.3%. The remaining fraction was not a partial success. It was the handful of records recent enough to fall inside a minutes-wide window, which the live stream would have captured anyway.

Bug two: acknowledging a delete for records that were never stored

Rejected records still counted. An auto-erase routine fired when the `out_of_range` count crossed 100, on the theory that a strap serving a hundred unusable records was holding a stale buffer worth clearing.

Given bug one, that condition was always true. Every backfill attempt produced a large `out_of_range` count, because every record produced one. The erase path fired on effectively every sync.

Worse, the acknowledgement was unconditional. The app sent the trim acknowledgement after processing a batch regardless of whether any record in that batch had actually been written. On 11 August 2026 one session rejected 160 records and acknowledged the trim anyway. The strap did what it was told and dropped them.

The promise inverted. Instead of "this is durable, you may forget it," the acknowledgement now meant "none of this could be read, please destroy it."

How it surfaced

Not as an error. As a night that showed 1.5 hours of sleep.

The phone had died overnight, which is the exact scenario backfill is built for. On reconnect, the strap should have replayed the missing hours. Instead the app rejected the replay, acknowledged the trim, and the strap cleared its buffer. By the time anyone looked, the record on the server was 1.5 hours, and that number was correct. It was an accurate report of everything that had survived.

The first instinct was a display bug, because 1.5 hours of sleep looks like a rendering problem and not like an integrity problem. It took querying the server directly to establish that the display was fine and the data was gone. That is the part worth internalising: a data-loss bug that lands cleanly presents as a cosmetic one.

A third defect, found while looking at the first two

Auditing what had made it through surfaced something separate. Of the rows that did land through the backfill path, 88% carried a heart rate below 20 bpm. That is not a plausible physiological value for a living adult, so those rows were parser output, not measurements.

This matters more than it looks. Every quality check on that table had been written to catch missing data. None had been written to catch present but impossible data. The rows existed, the timestamps looked fine, the row count went up. A dashboard built on "did data arrive" would have shown green.

What the numbers looked like

MEASURE	VALUE
Backfill records rejected	99.3%
Strap buffer depth	~72 hours
App accept window	Minutes
Auto-erase trigger	`out_of_range` ≥ 100, always true
Records rejected then acknowledged, single session	160
Backfilled rows with HR < 20 bpm	88%
Errors shown to the user across all of it	0

What actually went wrong, above the code

Each defect passes inspection in isolation. Validating a timestamp against an expected window is good practice. Clearing a buffer that keeps serving unusable records is reasonable hygiene. Acknowledging a batch after processing it is the normal shape of that loop.

The failure is that a destructive, irreversible, remote operation was made conditional on a counter rather than on a success. Nowhere in the path did anything ask the only question that mattered: *did a single record from this batch get written?* The acknowledgement was wired to control flow completing, not to work having been done.

The second failure is monitoring shaped around presence instead of plausibility. Row counts went up. Syncs completed. No exception was thrown at any point in the chain, because nothing here was an exception. The system was doing precisely what it had been told, quickly and often.

What replaced it

The accept window now derives from the strap's actual buffer depth rather than from a locally assumed gap, so a 69-hour-old record is an expected input instead of an anomaly. The acknowledgement is gated on a confirmed write count for the batch, and a batch with zero writes acknowledges nothing. Auto-erase no longer keys off a rejection counter at all, because a rejection counter measures the app's confusion rather than the strap's state.

A watchdog shipped alongside, in v157, and it checks plausibility rather than arrival: physiologically impossible values, backfill sessions with a zero write rate, and acknowledgements without a matching write. The specific bounds are not published here, because they are fitted to one device's firmware behaviour and would be misleading to copy.

What is still unknown

- How much history was lost in total. The destroyed records are gone from both sides, so the only honest answer is that the volume is unmeasurable by construction.
- Whether the parsing defect and the window defect share a root cause in timestamp handling, or are independent bugs that happened to live in the same file.
- Whether the 99.3% figure is stable across firmware versions. It was measured on one device over four days in August 2026, and no second device was available to reproduce it.

Where this generalises, and where it does not

The specific bugs are BLE-flavoured, but the shape is not. Any system that acknowledges receipt to a remote party that then discards its copy has the same exposure: message queues, log shippers, replication, offline-first sync, webhook consumers with an at-most-once delivery contract. If the acknowledgement is emitted from the same code path that runs whether or not the write succeeded, the bug is already present and waiting for a bad batch.

Where it does not generalise: if the remote side keeps its own durable copy regardless of acknowledgement, none of this is fatal, and the accept-window bug would have shown up as a retry storm instead of as loss. The destructive trim is what converts a sync failure into a permanent one.

The lesson

A silent failure that runs long enough stops being a bug and becomes the system's behaviour. This one ran with no error output, no alert, no failed assertion, and a row count that kept climbing. It was found because a number on a screen looked wrong, and the first three explanations for that number were all more comfortable than the real one.

Originally published at https://lucidailabs.com/essays-ble-silent-data-loss.

© 2026 Fabian Ilg. Licensed CC BY-NC-ND 4.0. Share it with attribution, do not sell it, do not publish altered versions.